

Computer Architecture & Concurrency – COMP0008

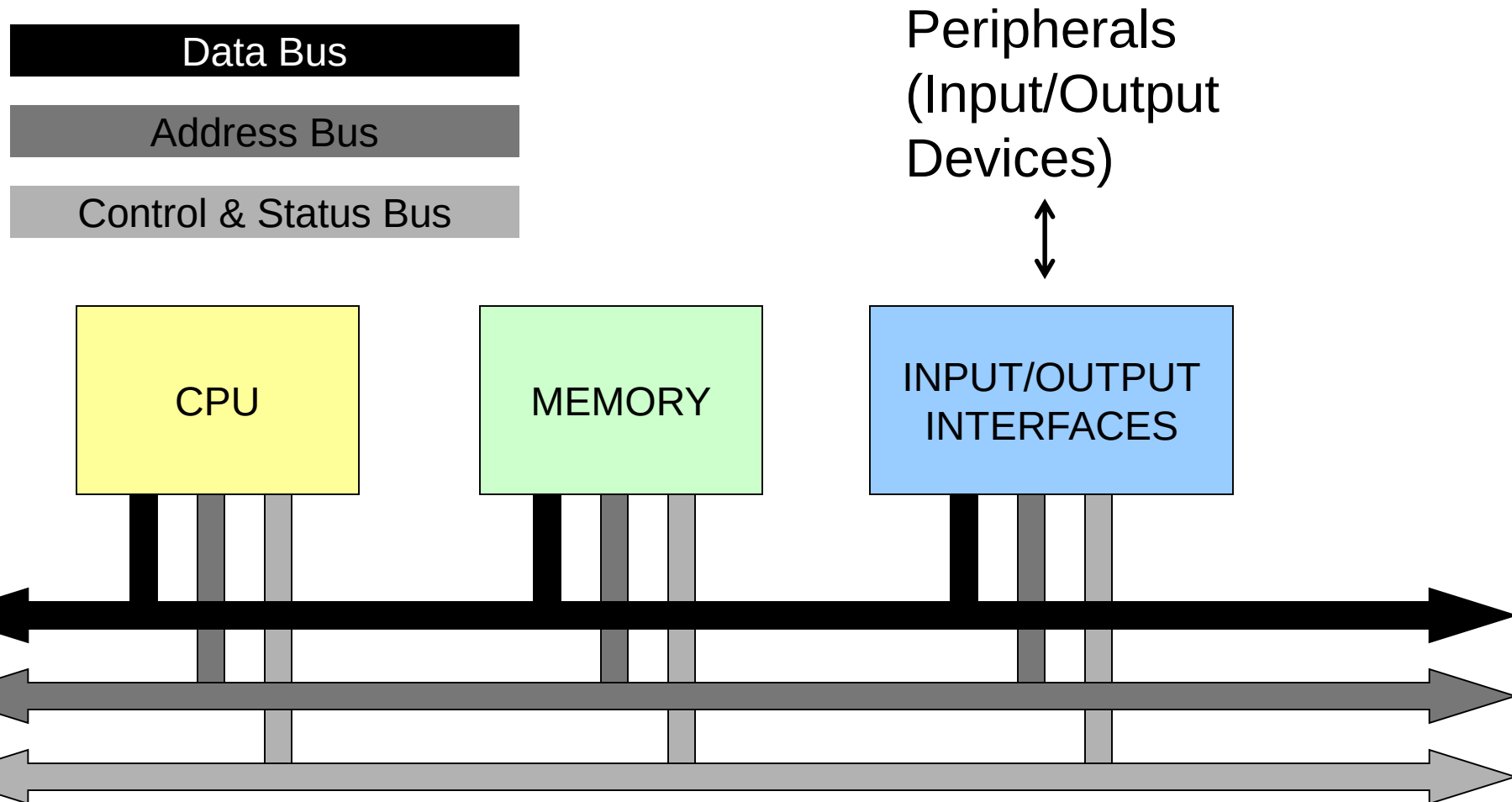
Lecture 2 Part 2: MIPS32 Hardware and Software

Kevin Bryson

K.Bryson@ucl.ac.uk

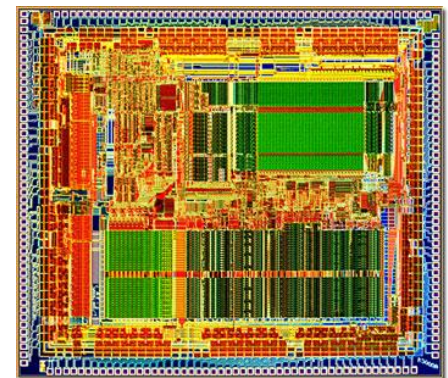
Darwin Building Room 632.

A processor's (or programmer's) abstract view of the computer architecture ...



So I guess we will be studying the PC / x86 ?

- Yes ... and No ...
- We will be studying the key concepts that most modern computer architectures are based upon.
- The Intel Core i3, i5, i7 processors in the PC are complex due to back-compatibility, so we will examine the **MIPS processor** which is much more eloquent design ... but the same concepts we learn will apply to *all* processors.



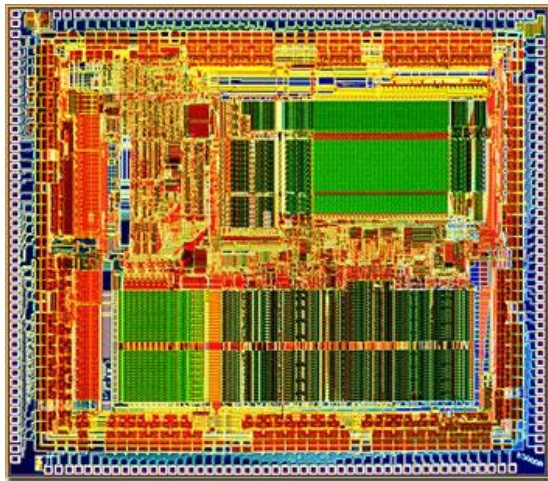
R3000 Processor (MIPS32)

Released: 1988

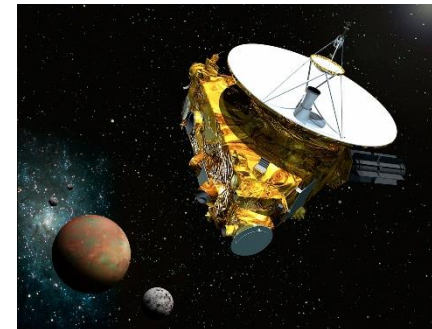
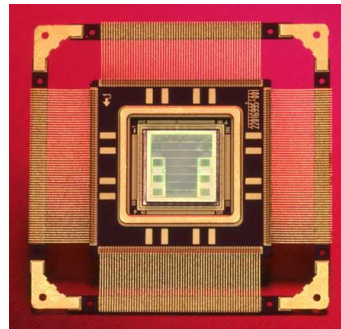
Transistor count: 115,000

Used in high-end workstations (e.g. Silicon Graphics SGI Personal IRIS 4D/20)

This processor was used to render 3D scenes in 90s movies like Jurassic Park or **Terminator 2** and more recently a radiation hardened version of the R3000 flew to pluto.

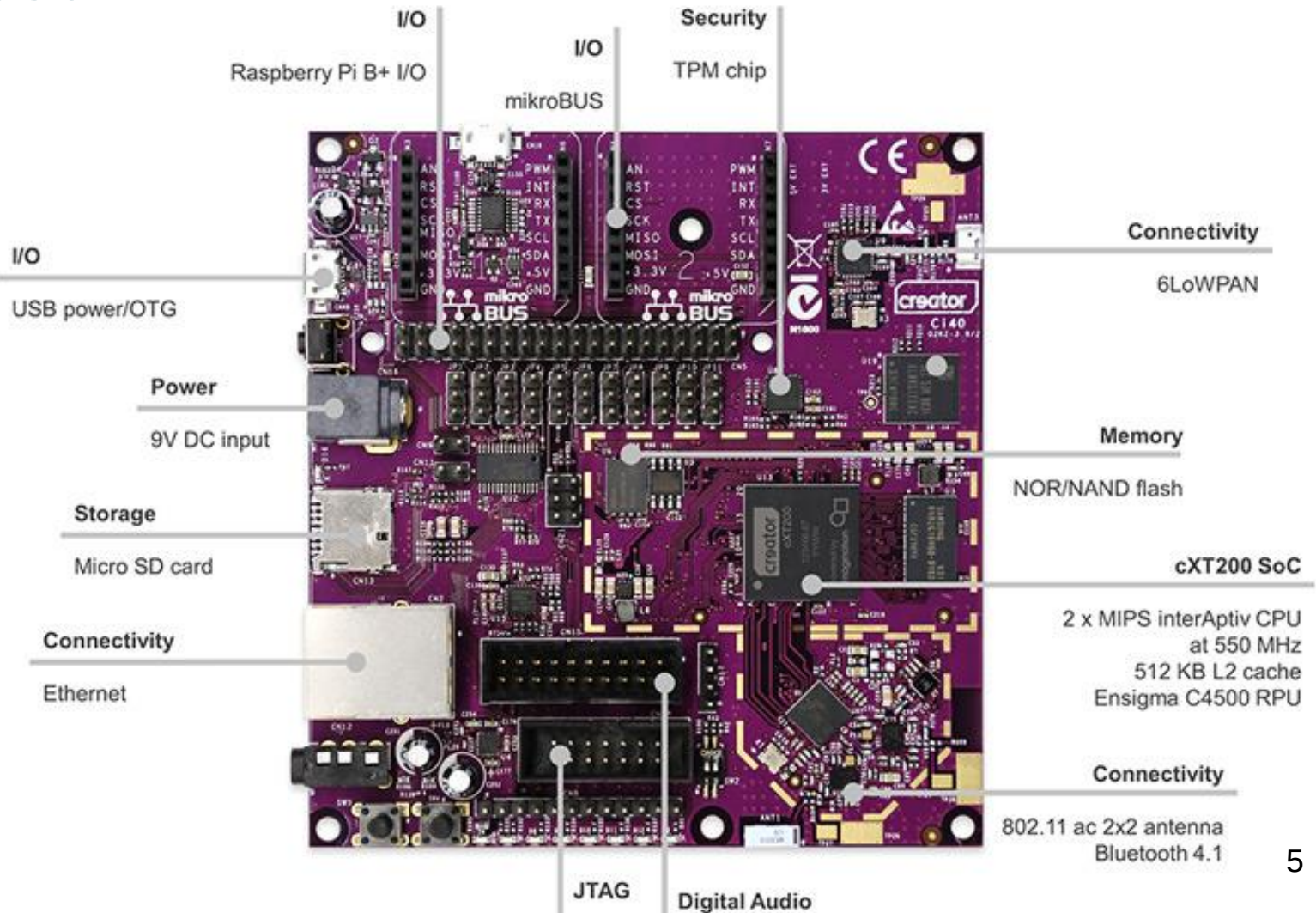


Silicon chip view of the R3000 processor

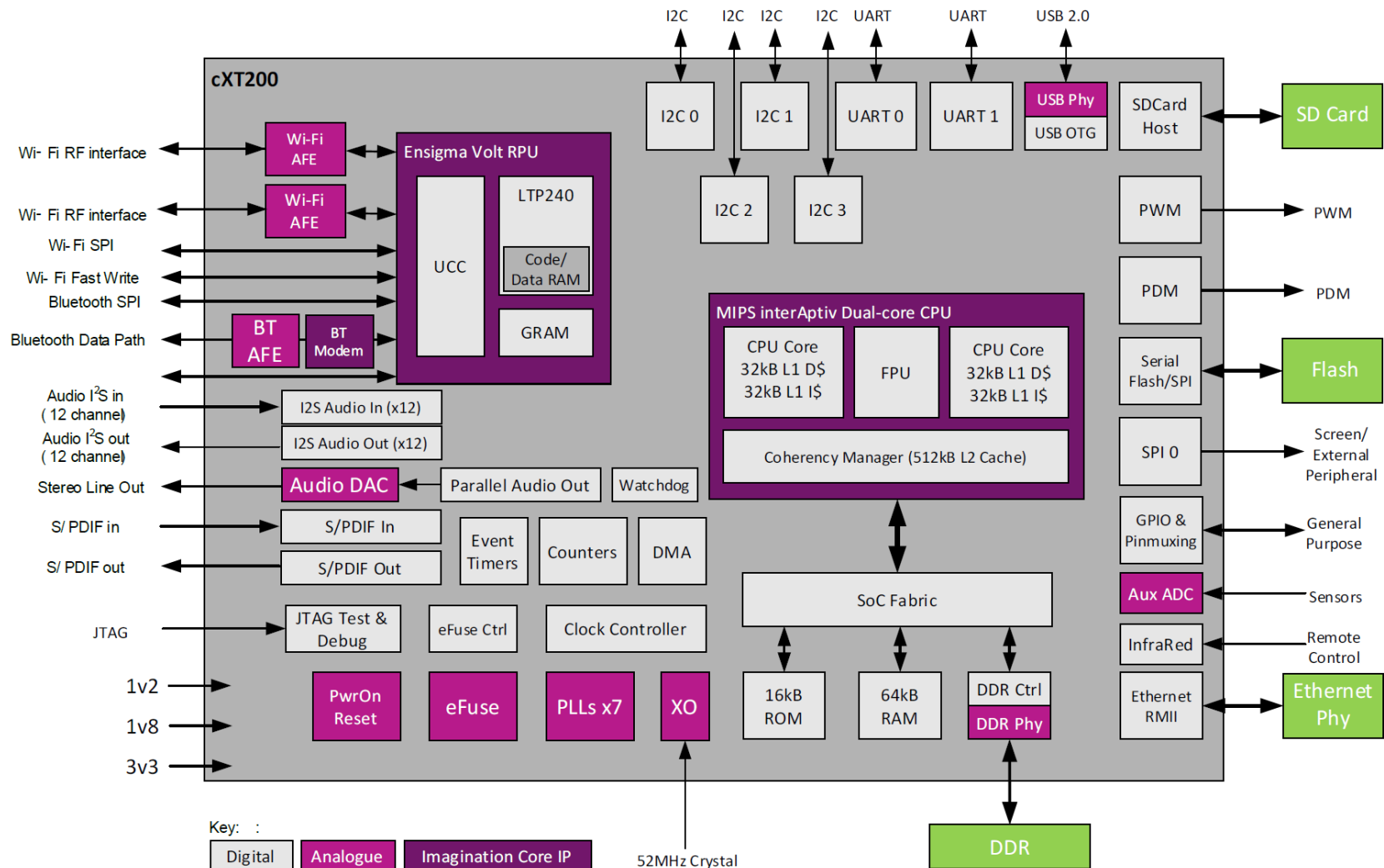


Radiation hardened Mongoose-V version of the R3000 processor used in the New Horizon space probe to Pluto.

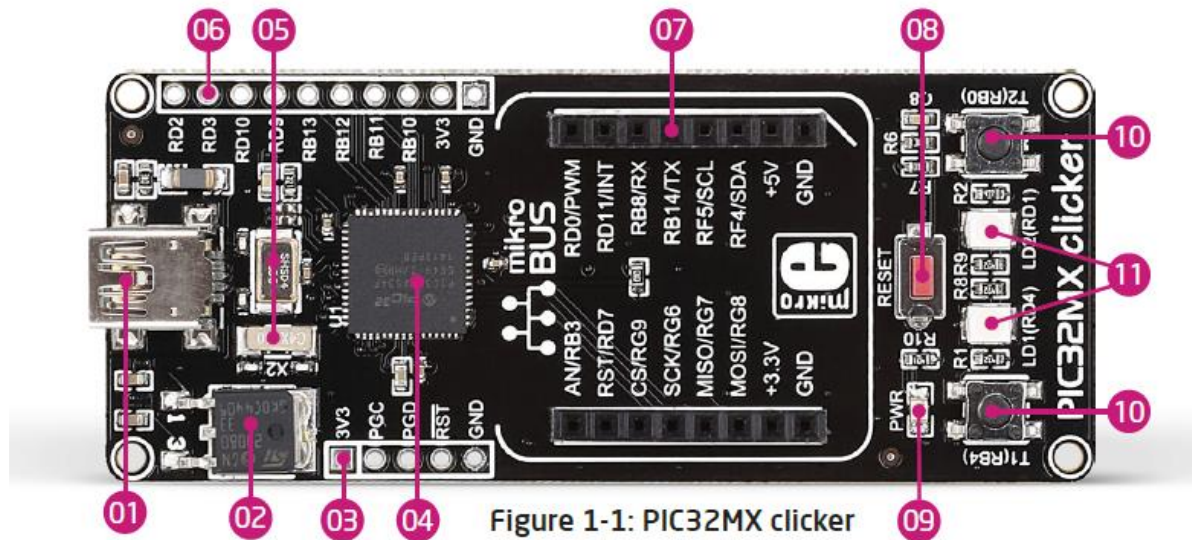
The Imaginations Creative Ci40 single board computer - like a Raspberry Pi but with a MIPS32 processor



cXT200 Dual Core MIPS32 Processor



Mikroelectronics “Clicker” microcontroller - like an Arduino but based on a PIC32 chip (MIPS32 architecture)



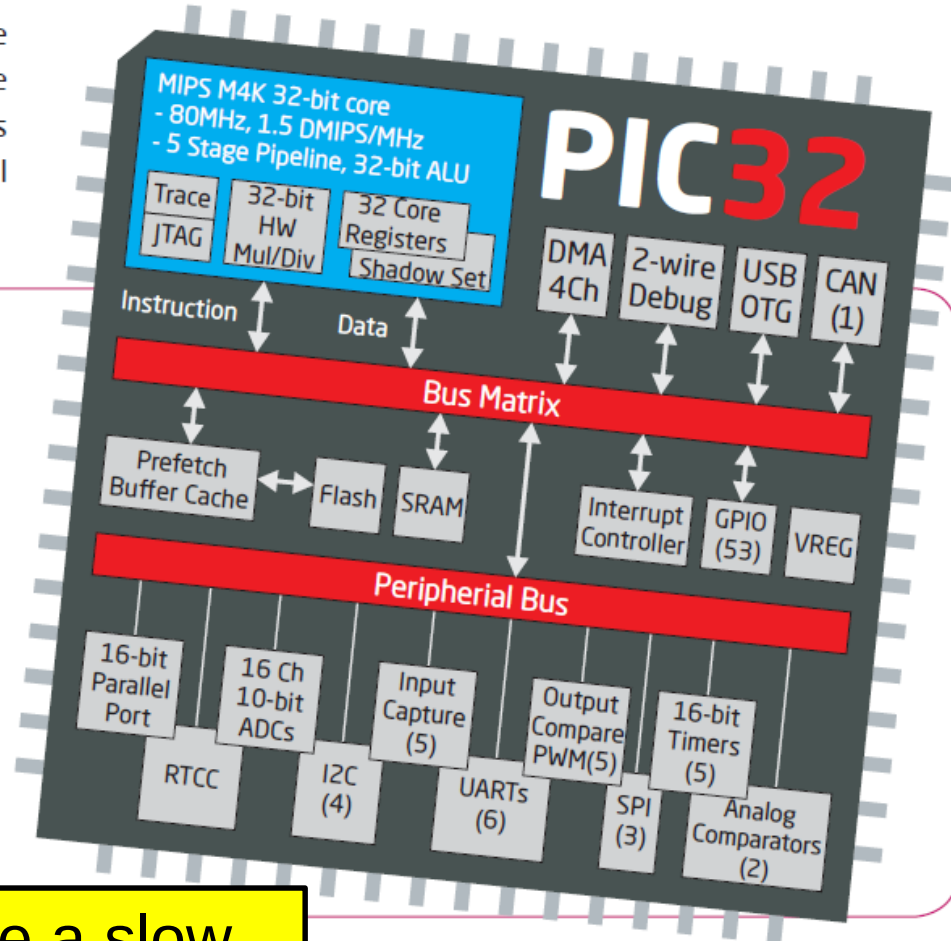
Microcontrollers generally do not run operating systems ... the processor directly runs the application code (programming to the bare metal)

PIC32 System on a Chip (SoC) structure

The PIC32MX clicker development tool comes with the **PIC32MX534F064H** microcontroller. This 32-bit MIPS M4K Core high performance microcontroller is rich with on-chip peripherals and features 64KB of Flash and 16KB RAM. It has integrated full speed USB 2.0. support.

Key microcontroller features

- 80MHz/105DMIPS, 32-bit MIPS M4K Core;
- 64KB Flash (plus 12K boot Flash);
- 16KB RAM (can execute from RAM);
- 53 I/O pins;
- SPI, I2C, A/D, CAN
- 16-bit Digital Timers;
- Internal Oscillator 8MHz, 32kHz;
- RTCC, etc.



Why would anyone want to use a slow microcontroller without much memory ?

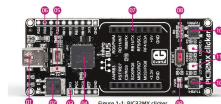
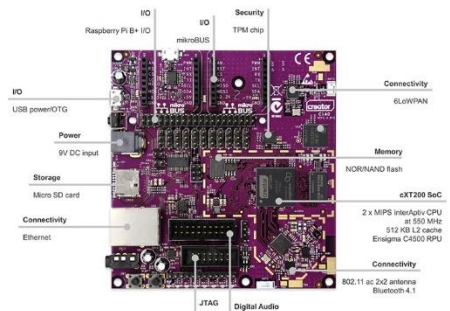
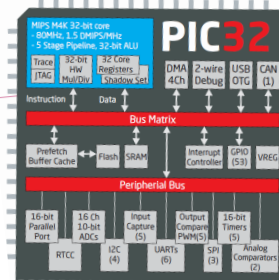
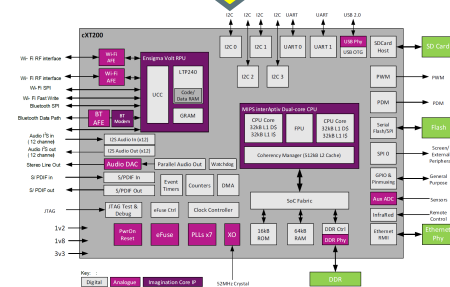
MIPS32 Instruction Set Architecture (ISA)

hides the 'hardware' from the software

```
add $s3,$s3,1 # MIPS assembly
```

```
001000 01011 01011 00000000000000001
```

MIPS32



Application Software		Programs
Operating Systems		Device Drivers
Architecture		Instructions Registers
Micro-architecture		Datapaths Controllers
Logic		Adders Memories
Digital Circuits		AND Gates NOT Gates
Analog Circuits		Amplifiers Filters
Devices		Transistors Diodes
Physics		Electrons

MARS Simulator is a MIPS32 Virtual Machine

C:\Users\ucackxb\Documents\teaching\2021_A5U_T1_COMP0008-Computer-Architecture-and-Concurrency\MARS_SIMULATOR\MARS_EXAMPLES\Fibonacci.asm - MARS 4.5

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Edit Execute

Fibonacci.asm

```
# Compute first twelve Fibonacci numbers and put in array, then print
.data
fibs: .word 0 : 12      # "array" of 12 words to contain fib values
size: .word 12          # size of "array"
.text
la $t0, fibs           # load address of array
la $t5, size            # load address of size variable
lw $t5, 0($t5)          # load array size
li $t2, 1              # 1 is first and second Fib. number
add.d $f0, $f2, $f4
sw $t2, 0($t0)          # F[0] = 1
sw $t2, 4($t0)          # F[1] = F[0] = 1
addi $t1, $t5, -2       # Counter for loop, will execute (size-2) times
loop: lw $t3, 0($t0)     # Get value from array F[n]
lw $t4, 4($t0)          # Get value from array F[n+1]
add $t2, $t3, $t4       # $t2 = F[n] + F[n+1]
sw $t2, 8($t0)          # Store F[n+2] = F[n] + F[n+1] in array
addi $t0, $t0, 4        # increment address of Fib. number source
```

Line: 1 Column: 1 Show Line Numbers

Mars Messages Run I/O

Clear

Registers Coproc 1 Coproc 0

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$s8	24	0x00000000
\$s9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffffc00
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x00400000
hi		0x00000000
lo		0x00000000

Understanding a modern computer ...

- Level 5) Problem Oriented Language Level

```
i = i + 1; // C source
```

- Level 4) Assembly Language Level

- Level 3) Operating System Level

```
add $s3,$s3,1 # MIPS assembly
```

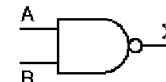
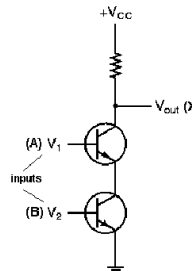
- Level 2) Instruction Set Architecture Level (ISA Level)

We will look at computer arithmetic and memory layout since the ISA level is entirely numbers.

```
001000 01011 01011 00000000000000001
```

- Level 1) Microprogramming Level
(not all computers have this level)

- Level 0.5) Modular view (datapath)
- Level 0) Digital Logic Level



A	B	X
0	0	1
0	1	1
1	0	1
1	1	0

